

Adventist University of Central Africa

Master's in Big Data Analytics

From One Tree to Many: Tree-Based Ensemble Learning

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics

Course Title: Data Mining

Course Code: MSDA 9124

Lecturer: Dr. Pacifique Nizeyimana

Student Name: Wayne Rubangisa

Student ID: 101220

September 2026

Contents

1	Introduction	1
2	Decision Trees as the Building Block	2
3	Ensemble Learning	3
4	Bagging	4
4.1	The Algorithm	4
4.2	Aggregation	5
5	Why Bagging Works	5
5.1	The Basic Intuition	5
5.2	The Variance of an Average	5
6	Random Forests	7
6.1	The Algorithm	7
6.2	Why Restricting Features Helps	8
6.3	Classification and Regression Forests	8
7	Bagging versus Random Forests	8
8	Out-of-Bag Estimation	9
8.1	The Probability of Being Left Out	10
8.2	Out-of-Bag Observations	10
9	Boosting	11
10	AdaBoost	11
10.1	The Intuition	12
10.2	The Algorithm	12
10.3	Interpreting the Equations	13
11	Gradient Boosting	13
11.1	Changing the Perspective	13
11.2	The Additive Model	14
11.3	Squared-Error Regression: The Simplest Case	14
11.4	Generalizing Beyond Squared Error	15
12	Why Boosting Works	15
13	Bagging versus Boosting	16

14 Random Forest versus Gradient Boosting	16
15 XGBoost and Modern Boosted Trees	19
16 A Unified Mathematical View	20
17 Practical Guidance	21
17.1 Key Hyperparameters	21
17.2 The Learning-Rate/Number-of-Trees Tradeoff	21
18 Conclusion	22

Abstract

Decision trees are simple, interpretable, and flexible predictive models, but a single tree is often unstable: small changes in the training data can produce very different trees, and deep trees tend to overfit. Ensemble methods address this instability by combining many trees into a single, more reliable predictor. This paper gives an intuitive yet technically rigorous introduction to the three dominant tree-ensemble paradigms: Bagging, Random Forests, and Boosting. We build the story from a single decision tree upward, explaining at each stage *what problem is being solved, how the mathematics solves it, and why the solution works*. Along the way we derive the variance-reduction identity that explains Bagging, clarify the precise relationship between Bagging and Random Forests, derive the AdaBoost weight update, show that Gradient Boosting is a form of functional gradient descent, and discuss how modern implementations such as XGBoost extend these ideas with regularization and second-order information. The paper closes with a unified mathematical view and practical guidance for choosing among these methods.

Keywords: decision trees, ensemble learning, bagging, random forests, boosting, AdaBoost, gradient boosting, XGBoost, bias-variance tradeoff

1 Introduction

Suppose you ask ten knowledgeable but imperfect friends to guess the price of a house. No single friend will be exactly right; each has blind spots, and each has seen a slightly different slice of the housing market. But if you average their guesses—or let them vote—the collective answer is often more reliable than any individual one. This is the central intuition behind *ensemble learning*: many imperfect models, combined correctly, can produce a model that is more accurate and more stable than any of its individual members.

This paper asks one recurring question, and returns to it again and again:

How can we turn many imperfect decision trees into a model that is more reliable than any individual tree?

Decision trees are a particularly good raw material for this kind of construction. They are:

- **Flexible:** a tree can approximate almost any function given enough splits.
- Capable of modeling **nonlinear relationships** without requiring the user to specify a functional form in advance.
- Naturally able to capture **interactions** between features, since a split deep in the tree is implicitly conditioned on all the splits above it.

- **Unstable:** small changes in the training data can produce a substantially different tree structure.

That last property—instability, otherwise known as *high variance*—sounds like a weakness. In an ensemble, it becomes an asset. If a model changes noticeably every time we perturb the training data, then training it on many different perturbed versions of the data will produce many genuinely different trees. As we will see in Section 5, having genuinely different (not just superficially different) models is exactly what makes averaging effective.

This paper is organized around two families of ensemble strategies, distinguished by *how the individual trees relate to one another*:

- **Bagging and Random Forests:** the trees are grown independently and in parallel, then their predictions are aggregated.
- **Boosting** (AdaBoost, Gradient Boosting, XGBoost): the trees are grown sequentially, and each new tree is trained to correct the mistakes of the ensemble built so far.

We will keep returning to a simple four-sentence story as scaffolding:

A single tree makes a prediction. Bagging builds many trees independently and aggregates them. Random Forest adds feature-level randomness to decorrelate the trees. Boosting builds trees sequentially, with later trees improving the existing model.

By the end of the paper, every equation we introduce should feel like a precise restatement of something in that story, rather than an obstacle standing between the reader and understanding.

2 Decision Trees as the Building Block

Before combining trees, we need a clear (if brief) picture of what a single tree is doing. A decision tree makes predictions by asking a sequence of simple yes/no questions about the input features.

- A **node** represents a question about a feature, such as “is square footage less than 1500?”
- A **split** is the rule that sends an observation to the left or right child node based on the answer to that question.
- A **feature** is one of the input variables (square footage, number of bedrooms, zip code, and so on) that the tree is allowed to split on.
- A **leaf** is a terminal node with no further splits; it holds the tree’s final prediction for any observation that reaches it.

For **regression**, the prediction at a leaf is typically the average of the training outcomes that fall into that leaf. For **classification**, the prediction is typically the majority class (or the class proportions, if we want a probability) among the training observations in that leaf.

Trees are grown by repeatedly choosing the split that most improves some measure of fit—for example, the reduction in squared error for regression, or the reduction in Gini impurity or entropy for classification. If we let the tree keep splitting, it will eventually create a leaf for every training point, achieving zero training error. This is precisely the problem: a tree that is deep enough to fit the training data perfectly has almost certainly memorized noise along with signal. Such a tree will typically generalize poorly to new data—the classic symptom of **overfitting**.

There is a second, closely related problem that will matter enormously for the rest of this paper. Suppose we train a tree, then remove a handful of observations from the training set (or add a few new ones) and train again. Because the very first split is chosen to be the single best split over the *entire* dataset, a small change in the data can tip the balance toward a different first split—and once the first split changes, every split below it in the tree can change as well. The tree is not making a small, proportional adjustment; it can restructure itself substantially. This sensitivity to the training data is what we mean by saying that decision trees have **high variance**: if we imagine repeating the training process on many different samples from the same underlying population, the resulting trees (and their predictions) can vary a great deal.

High variance is bad news for a single tree used in isolation. But notice what it implies: if we train many trees on many different (but related) versions of the data, we will get many genuinely different trees, each capturing a slightly different aspect of the underlying pattern. This is exactly the raw material that ensembles need, and it is the motivation for the method we introduce next: Bagging.

3 Ensemble Learning

An **ensemble** is a predictive model built by combining the predictions of multiple individual models, called **base learners**. Ensemble learning is the general strategy of constructing and combining such models rather than relying on a single one.

Two ingredients determine whether an ensemble will outperform its individual members:

1. **Diversity**: the base learners should make different kinds of errors. If every model in the ensemble makes exactly the same mistakes, combining them buys us nothing.
2. **Aggregation**: there must be a sensible way to combine the individual predictions—most commonly **averaging** for regression outputs or continuous scores, and **voting** for class labels.

The key insight, which we will make precise in Section 5, is that ensembles help most when the errors of the individual models are not perfectly correlated. If ten friends all read the same (possibly wrong) real-estate report before guessing a house price, averaging their guesses will not help much, because they are all wrong in the same direction. If instead each friend forms their opinion from a different, independent source of information, their errors will tend to partially cancel when averaged. This single idea—*combine models whose errors are not perfectly correlated*—is the thread that connects Bagging, Random Forests, and (in a different way) Boosting.

4 Bagging

Bagging stands for **B**ootstrap **A**ggregating, introduced by Breiman [1]. The idea directly addresses the high-variance problem identified in Section 2: if a model is unstable under small perturbations of the training data, why not deliberately create many such perturbed datasets, train one model on each, and average the results?

4.1 The Algorithm

Bagging proceeds in four steps.

1. Start with a training dataset $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$ of n observations.
2. Generate B **bootstrap samples**, $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(B)}$, each of size n , by **sampling with replacement** from $\mathcal{D}$. “With replacement” means that after an observation is drawn into the sample, it is placed back into the pool and can be drawn again.
3. Train one model $\hat{f}^{(b)}$ on each bootstrap sample $\mathcal{D}^{(b)}$, independently of the others.
4. Aggregate the B predictions into a single ensemble prediction.

Example 4.1 (Sampling with replacement). *Suppose our training set has five observations, indexed $\{1, 2, 3, 4, 5\}$. A bootstrap sample of size five drawn with replacement might look like:*

$$\mathcal{D}^{(1)} = \{3, 1, 3, 5, 2\}.$$

Notice that observation 3 appears twice, while observation 4 does not appear at all. This is expected, not an error: because we sample with replacement, each draw is independent of the others, so some points are naturally selected more than once and others are naturally left out entirely. A second bootstrap sample, drawn independently, might be $\mathcal{D}^{(2)} = \{2, 2, 4, 1, 4\}$ —a different subset with different repeats. Each bootstrap sample is a slightly different “view” of the same underlying data, which is exactly what we want if we are trying to grow trees that disagree with one another in useful ways.

4.2 Aggregation

For **regression**, we average the predictions of the B models:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x). \quad (1)$$

This equation is saying something very simple: ask every model for its prediction at the point x , and report the average of those B numbers as the ensemble’s prediction. Nothing more mysterious is happening here than averaging ten friends’ house-price guesses.

For **classification**, the most common aggregation rule is **majority voting**: each model $\hat{f}^{(b)}$ votes for a class, and the ensemble predicts whichever class received the most votes,

$$\hat{f}_{\text{bag}}(x) = \arg \max_k \sum_{b=1}^B \mathbf{1}[\hat{f}^{(b)}(x) = k], \quad (2)$$

where $\mathbf{1}[\cdot]$ is the indicator function, equal to 1 if its argument is true and 0 otherwise. When each base learner instead outputs class *probabilities* rather than a hard label, an alternative—and often better-behaved—aggregation rule is to average the predicted probabilities across models and then take the class with the highest average probability. Both approaches implement the same idea: let every model weigh in, and let the crowd’s consensus be the final answer.

5 Why Bagging Works

This is the section where Bagging changes from “a plausible-sounding trick” into “a method whose benefits we can actually quantify.”

5.1 The Basic Intuition

If several noisy estimates of the same quantity are averaged, the noise can partially cancel out, because on any given draw some estimates will be too high and others too low. Averaging does not remove any single model’s bias, but it can substantially reduce the spread—the **variance**—of the combined prediction. This is the mechanism Bagging exploits: individual trees are low-bias, high-variance predictors, and averaging is a direct tool for reducing variance.

5.2 The Variance of an Average

Let $X_1, \dots, X_B$ denote the predictions of B base learners at a fixed input point x (we suppress the dependence on x to lighten the notation). Suppose these predictions are

approximately identically distributed, each with variance

$$\text{Var}(X_b) = \sigma^2, \quad b = 1, \dots, B,$$

and suppose each pair of predictions has the same pairwise correlation $\rho = \text{Corr}(X_b, X_{b'})$ for $b \neq b'$. We want to know the variance of the average,

$$\bar{X} = \frac{1}{B} \sum_{b=1}^B X_b.$$

Expanding the variance of a sum,

$$\text{Var}(\bar{X}) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(X_b) + \sum_{b \neq b'} \text{Cov}(X_b, X_{b'}) \right].$$

There are B diagonal terms, each equal to σ^2 , and $B(B-1)$ off-diagonal terms, each equal to $\rho\sigma^2$ (since $\text{Cov}(X_b, X_{b'}) = \rho\sigma^2$ by definition of correlation). Substituting,

$$\text{Var}(\bar{X}) = \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] = \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2.$$

A small amount of algebra puts this in the more illuminating form

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B X_b\right) = \sigma^2 \left(\rho + \frac{1-\rho}{B} \right). \quad (3)$$

Here is the important part. Equation (3) tells us exactly how much averaging helps, and the answer depends critically on the correlation ρ between the individual models:

- If $\rho = 1$ (the models are perfectly correlated—they always make the same errors in the same direction), the formula reduces to $\text{Var}(\bar{X}) = \sigma^2$: no benefit at all from averaging, no matter how large B is.
- If $\rho = 0$ (the models' errors are uncorrelated), the formula reduces to $\text{Var}(\bar{X}) = \sigma^2/B$: the variance shrinks toward zero as we add more models.
- For intermediate values of ρ , the second term $\sigma^2(1-\rho)/B$ can be driven to zero by increasing B , but the first term $\rho\sigma^2$ is a floor that no amount of additional averaging can remove.

This is the single most important quantitative fact in this paper, so it is worth restating in words: *averaging reduces variance, but only the part of the variance that is not shared across models*. Adding more trees to a bagged ensemble helps, but the benefit saturates, and the ceiling on that benefit is set by how correlated the trees are with one another—not by how many trees we grow. This observation motivates Random Forests, whose

entire design is aimed at pushing ρ down.

Remark 5.1. *Bagging is best understood as primarily a variance-reduction technique. It does very little to fix a base learner that is systematically biased: if every bootstrap-trained tree shares the same blind spot, averaging them will not remove that blind spot, since bias is a property of $\mathbb{E}[X_b]$, not of the spread around it. This is why Bagging is typically applied to low-bias, high-variance base learners such as deep, largely unpruned decision trees.*

6 Random Forests

Equation (3) leaves us with a clear engineering target: reduce the correlation ρ between trees without inflating each tree’s individual variance σ^2 too much. Bootstrap sampling alone provides only a limited amount of decorrelation, because every bootstrap sample is drawn from the same training set, and the very best split at the top of the tree is usually determined by whichever few features are most predictive overall—so bagged trees tend to still look fairly similar to one another near the root. **Random Forests**, introduced by Breiman [2], address this directly by adding a second, independent source of randomness: restricting which features each split is even allowed to consider.

6.1 The Algorithm

A Random Forest combines three ingredients:

1. **Bootstrap sampling of observations**, exactly as in Bagging: each tree is grown on an independent bootstrap sample of the training data.
2. **Random feature selection at each candidate split**: rather than choosing the best split by searching over all p available features, the algorithm first draws a random subset of $m < p$ features (a typical default is $m \approx \sqrt{p}$ for classification or $m \approx p/3$ for regression) and restricts the search for the best split to only that subset.
3. **Aggregation across many trees**, exactly as in Bagging: average the predictions for regression, or take a majority vote for classification.

If we write $T_b(x)$ for the prediction of the b -th tree, the Random Forest prediction is

$$\hat{f}_{\text{RF}}(x) = \text{Aggregate}(T_1(x), \dots, T_B(x)), \quad (4)$$

where $\text{Aggregate}(\cdot)$ denotes averaging for regression or majority voting (or probability averaging) for classification—the same aggregation rules used in Bagging.

6.2 Why Restricting Features Helps

Here is the intuition. Imagine a dataset with twenty features, of which only two or three are strongly predictive. If every tree is allowed to consider all twenty features at every split, nearly every tree will choose the same one or two dominant features for its earliest, most influential splits—the trees will look structurally similar to one another, and the correlation ρ in Equation (3) will stay high. **This is where Random Forest differs from ordinary Bagging.** By forcing each split to ignore most of the available features and choose only among a random subset, we occasionally prevent a tree from using its favorite dominant feature at a given split, forcing it to rely on a different, otherwise-overshadowed feature instead. Repeated across many trees and many splits, this produces trees that are genuinely more different from one another in structure—lower ρ —at a modest cost in the individual accuracy σ^2 of each tree. Equation (3) tells us this exchange is usually a good trade: a small increase in each tree’s own variance is often more than repaid by a larger decrease in the shared correlation between trees.

6.3 Classification and Regression Forests

The recipe above applies to both tasks; only the leaf-level prediction and the aggregation rule change. In a **regression forest**, each $T_b(x)$ is a real number (the average training outcome in the relevant leaf), and Equation (4) becomes a simple average, just as in Equation (1). In a **classification forest**, each $T_b(x)$ is a predicted class (or a vector of class probabilities), and Equation (4) becomes majority voting or probability averaging, just as in Equation (2).

7 Bagging versus Random Forests

This section exists to remove any ambiguity between two ideas that are frequently, and incorrectly, treated as synonyms.

The simplest possible statement of the distinction is this: **Bagging is a general ensemble strategy; Random Forest is a specific tree-based algorithm that uses Bagging plus random feature selection.** Bagging does not inherently require decision trees at all—one could, in principle, bag linear regressions, bag neural networks, or bag any other unstable base learner, by following exactly the four steps of Section 4 with a different model in step 3. Random Forest is a narrower, tree-specific procedure that happens to use Bagging as one of its two sources of randomness.

Table 1 lays out the comparison explicitly.

A useful (though informal) shorthand is:

Random Forest = Bagging applied to decision trees + additional

Table 1: Bagging versus Random Forests.

	Bagging	Random Forest
Scope	General ensemble framework	Specific tree-based algorithm
Base learner	Any model (trees, linear models, neural nets, ...)	Decision trees only
Observation sampling	Bootstrap samples, sampled with replacement	Bootstrap samples, sampled with replacement
Feature sampling	None; every split may use every feature	Random subset of features considered at every split
Source(s) of randomness	One: resampling of observations	Two: resampling of observations <i>and</i> restriction of features
Aggregation	Averaging (regression) or voting (classification)	Averaging (regression) or voting (classification)
Typical correlation between models	Higher (models tend to resemble each other more)	Lower (feature-level randomness further decorrelates trees)

randomness at each split from restricting the candidate features.

This shorthand captures the core relationship well, though it is worth noting that concrete implementations and formal definitions can differ in details—for instance, in how the feature subset size m is chosen, or in whether trees are grown to full depth or lightly regularized.

Addressing a common misconception directly. A question that arises naturally at this point is: “*Is Random Forest just Bagging?*” The answer is: it is closely related, but not exactly the same. A Random Forest uses the Bagging idea (bootstrap sampling plus aggregation) as its foundation, but it introduces an additional, independent randomization mechanism—random feature selection at each split—that Bagging by itself does not include. As Section 5 showed mathematically, this extra randomization exists for a specific, quantifiable reason: it further reduces the correlation ρ between trees, which Equation (3) shows is the main lever for reducing the variance of the ensemble beyond what plain Bagging can achieve.

Figure 1 illustrates the structural difference: in Bagging, each tree may split on any feature; in a Random Forest, each split is chosen from only a random subset of the available features (shaded).

8 Out-of-Bag Estimation

Bootstrap sampling has a convenient side effect: because each bootstrap sample is drawn with replacement from n observations, it will typically miss some of the original observations entirely. Those left-out observations turn out to be useful for free.

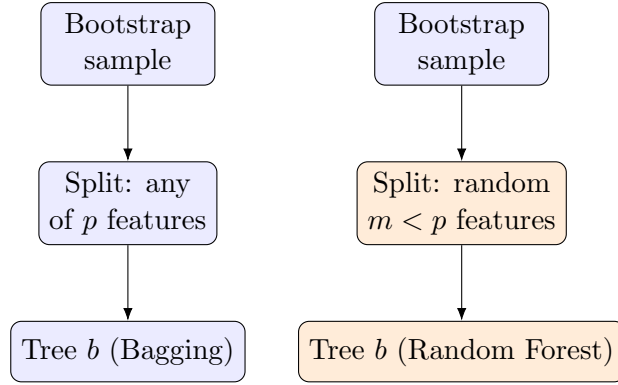


Figure 1: In Bagging (left), every split may consider all p features, so trees across bootstrap samples tend to resemble each other near the root. In a Random Forest (right), each split is restricted to a random subset of $m < p$ features, which further decorrelates the resulting trees.

8.1 The Probability of Being Left Out

Consider a single observation i and a single bootstrap sample of size n drawn (with replacement) from the original n training points. On each of the n draws, the probability that observation i is *not* selected is $(1 - 1/n)$. Since the n draws are independent, the probability that observation i is excluded from the *entire* bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n.$$

As n grows large, this expression converges to a well-known limit,

$$\left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368.$$

So as the sample size becomes large, each observation has approximately a 36.8% chance of being excluded from a given bootstrap sample, and therefore approximately a 63.2% chance of appearing in it (possibly more than once, as in Example 4.1).

8.2 Out-of-Bag Observations

For any given tree b , the observations that were *not* included in that tree’s bootstrap sample are called that tree’s **out-of-bag (OOB) observations**. Because tree b never saw these observations during training, they act exactly like a held-out validation set—but one that comes for free, without sacrificing any training data or requiring an explicit train/validation split.

To estimate the generalization performance of the full ensemble, we predict each training observation i using only the subset of trees for which i was out-of-bag, and

average or vote over that subset:

$$\hat{f}_{\text{OOB}}(x_i) = \text{Aggregate} \left(\{T_b(x_i) : \text{observation } i \text{ is out-of-bag for tree } b\} \right).$$

Comparing these out-of-bag predictions to the true labels y_i across all n training observations yields the **out-of-bag error**, a nearly unbiased estimate of the ensemble’s test error that requires no separate validation set and no cross-validation loop—each observation, in effect, validates the (roughly 37% of) trees that never trained on it.

9 Boosting

Bagging and Random Forests both grow trees *independently*: every tree could in principle be trained on a different machine at the same time, and none of the trees know anything about what the others are doing. **Boosting** takes the opposite approach.

Bagging: “Build models independently and combine them.”

Boosting: “Build models sequentially, with each new model attempting to improve the current ensemble.”

The base learners in boosting are typically deliberately simple—often shallow trees with only a handful of splits, sometimes even single-split “decision stumps.” Such a model is called a **weak learner**: one that performs only slightly better than random guessing on its own. The premise of boosting is that even a collection of weak learners can be combined into a strong learner, *provided* each new learner is trained to focus specifically on the mistakes of the learners that came before it.

The high-level structure is a chain of corrections:

$$\text{Model}_1 \longrightarrow \text{errors} \longrightarrow \text{Model}_2 \longrightarrow \text{remaining errors} \longrightarrow \text{Model}_3 \longrightarrow \dots$$

It is worth being precise about what this chain does *not* mean. Boosting does not simply retrain a single tree repeatedly hoping for a better one; nor does model 2 replace model 1. Instead, boosting constructs an **additive model**, a running sum of many weak learners, where each new term is chosen specifically to patch the errors left behind by the sum of all previous terms. We will make this precise first for classification (AdaBoost) and then for general loss functions (Gradient Boosting).

10 AdaBoost

AdaBoost (Adaptive Boosting), introduced by Freund and Schapire [3], was the first boosting algorithm to achieve widespread practical success and remains the clearest entry point into the boosting idea.

10.1 The Intuition

Train a weak classifier on the data. Some observations will be classified correctly, and some incorrectly. Rather than treating every observation equally in the next round, AdaBoost **turns up the volume** on the observations the current model got wrong: it increases their weight, so that the next weak learner is trained on a version of the problem where getting those particular observations right matters more. Observations that are already classified correctly have their weight reduced (or simply left relatively smaller), since the model does not need to spend further effort on them. Over many rounds, this forces successive learners to specialize in the parts of the input space that previous learners found difficult.

10.2 The Algorithm

Assume a binary classification problem with labels $y_i \in \{-1, +1\}$. Each training observation i carries a weight $w_i^{(m)}$ at round m , initialized uniformly: $w_i^{(1)} = 1/n$ for all i . At each round $m = 1, \dots, M$:

1. Fit a weak classifier $h_m(x)$ to the training data, using the current weights $w_i^{(m)}$ (observations with larger weight count more toward the fitting criterion).
2. Compute the weighted error rate of this classifier,

$$\epsilon_m = \frac{\sum_{i=1}^n w_i^{(m)} \mathbf{1}[y_i \neq h_m(x_i)]}{\sum_{i=1}^n w_i^{(m)}}. \quad (5)$$

In words, this is simply the fraction of (weighted) training observations that h_m gets wrong.

3. Compute the learner's voting weight,

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right). \quad (6)$$

4. Update each observation's weight for the next round,

$$w_i^{(m+1)} \propto w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i)), \quad (7)$$

then renormalize so that the weights sum to one.

After M rounds, the final classifier is a weighted vote of all the weak learners,

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right).$$

10.3 Interpreting the Equations

Don't let Equation (6) intimidate us. The quantity α_m is just a number that measures how much we should trust learner m 's vote, and its shape tells a very sensible story:

- If ϵ_m is small (the learner is highly accurate, say $\epsilon_m = 0.1$), then $(1 - \epsilon_m)/\epsilon_m$ is large, its logarithm is a large positive number, and α_m is large: an accurate learner gets a loud voice in the final vote.
- If $\epsilon_m = 0.5$ (the learner is exactly as good as a coin flip), then $(1 - \epsilon_m)/\epsilon_m = 1$, its logarithm is zero, and $\alpha_m = 0$: a useless learner is given no voice at all.
- If ϵ_m is larger than 0.5 (the learner is worse than random guessing), α_m becomes negative, effectively flipping the learner's vote—a systematically wrong learner is still informative, just in reverse.

Now look at Equation (7), the weight update. The term $y_i h_m(x_i)$ is $+1$ when the classifier gets observation i right (matching signs) and -1 when it gets observation i wrong (opposite signs). Substituting these two cases:

- Correctly classified: the exponent is $-\alpha_m$ (a negative number, assuming $\alpha_m > 0$), so the weight is *multiplied by a factor less than one*—the weight shrinks.
- Incorrectly classified: the exponent is $+\alpha_m$, so the weight is *multiplied by a factor greater than one*—the weight grows.

This is exactly the intuition we described in words at the start of the section: observations the current ensemble gets wrong become more heavily weighted for the next round, forcing the next weak learner to pay more attention to them, while observations that are already handled well fade into the background.

11 Gradient Boosting

AdaBoost is elegant, but its reweighting scheme is specific to classification with exponential-style loss. **Gradient Boosting**, formalized by Friedman [4], reframes boosting in a way that works for essentially any differentiable loss function, covering regression, classification, and beyond, within a single unified recipe.

11.1 Changing the Perspective

Instead of explicitly reweighting misclassified observations, Gradient Boosting asks a more general question at each round: *in what direction should the current model's predictions change, at each training point, in order to reduce the loss function the fastest?* Each new weak learner is trained to approximate that direction of improvement, and is then added to the running model.

11.2 The Additive Model

Let $F_m(x)$ denote the ensemble’s prediction after m rounds of boosting. Gradient Boosting builds this sequence by adding one new weak learner at a time,

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad (8)$$

where:

- $F_{m-1}(x)$ is the model built after the previous round—the current, imperfect prediction.
- $h_m(x)$ is a newly trained weak learner (typically a shallow regression tree).
- $\eta \in (0, 1]$ is the **learning rate** (also called the **shrinkage** parameter), which controls how much of the new learner’s suggestion we actually incorporate.

The learning rate embodies a genuine tradeoff. A larger η makes faster progress per round but risks overshooting and overfitting; a smaller η makes more cautious, incremental progress, typically requiring more rounds M to reach the same level of fit, but often generalizing better because no single tree is allowed to dominate the final model.

11.3 Squared-Error Regression: The Simplest Case

To see concretely what “the direction of improvement” means, consider regression with squared-error loss,

$$L(y, F(x)) = \frac{1}{2}(y - F(x))^2. \quad (9)$$

(The factor of $\frac{1}{2}$ is a bookkeeping convenience that will cancel neatly in a moment.) We want to know how the loss changes as we adjust $F(x)$, which means differentiating with respect to $F(x)$:

$$\frac{\partial L(y, F(x))}{\partial F(x)} = -(y - F(x)).$$

The direction that most rapidly *decreases* the loss is the negative of this gradient,

$$-\frac{\partial L(y, F(x))}{\partial F(x)} = y - F(x). \quad (10)$$

Here is the important part. The right-hand side of Equation (10) is just the **residual**: the difference between the true value y and the model’s current prediction $F(x)$. For squared-error loss, then, “the direction in which the model should change to reduce the loss” is nothing more exotic than “the amount by which the model is currently wrong.” This gives us an unusually clean and intuitive special case of Gradient Boosting: at each round, fit a new tree $h_m(x)$ to predict the current residuals $y_i - F_{m-1}(x_i)$, then add a shrunk copy of that tree to the running model via Equation (8). **Think of the next tree as a correction**: it is not trying to predict y from scratch, but only trying to

predict the part of y that the current ensemble has not yet explained.

11.4 Generalizing Beyond Squared Error

The residual-fitting picture above is a special case of a more general procedure. For an arbitrary differentiable loss function $L(y, F(x))$ —for example, the logistic loss used for classification—the same logic applies, but the quantity we fit at each round is the **negative gradient** of the loss with respect to the current predictions, evaluated at the training points,

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F(x)=F_{m-1}(x)}, \quad i = 1, \dots, n.$$

These quantities $r_i^{(m)}$ are called **pseudo-residuals**: for squared-error loss they coincide exactly with the ordinary residuals of Equation (10), and for other loss functions they play the analogous role of “the direction each prediction should move to reduce the loss.” A new tree $h_m(x)$ is trained to approximate these pseudo-residuals, and Equation (8) is used to fold it into the ensemble. This is why the method is called *Gradient Boosting*: at every round, we are taking a small step in a function space, in the direction indicated by a gradient, exactly as gradient descent takes a small step in parameter space in the direction indicated by a derivative.

12 Why Boosting Works

Bagging was framed, in Section 5, primarily as a variance-reduction device applied to low-bias, high-variance base learners. Boosting is best understood from a different angle: it starts from weak learners that individually have relatively high *bias* (they are too simple to fit the data well on their own) and reduces that bias by sequentially adding corrections, each aimed at the errors the ensemble has not yet resolved. With every additional round, the ensemble can represent an increasingly rich function, so its bias tends to fall as boosting proceeds.

This does not mean boosting is a free lunch. Several knobs control the tradeoff between fitting the training data well and generalizing to new data:

- **Number of trees (M)**: more rounds allow the model to fit the training data more closely, but past some point this starts fitting noise rather than signal.
- **Tree depth**: deeper trees per round make each individual correction more powerful (and more prone to overfitting on its own), while shallow trees make more conservative, incremental corrections.
- **Learning rate (η)**: smaller values make each round contribute less, generally requiring more rounds but often improving generalization—a pattern sometimes

summarized as “shrinkage plus more trees beats large steps and fewer trees.”

- **Loss function:** choosing a loss appropriate to the task (squared error, absolute error, logistic loss, and so on) determines what the pseudo-residuals actually represent and how sensitive the method is to outliers.
- **Regularization:** explicit penalties on tree complexity (discussed further in Section 15) can be added to discourage the ensemble from growing overly intricate corrections.

The common intuition that “more trees is always better” is not correct for boosting: unlike Bagging, where adding more independent trees to the average can only help (Equation (3) shows the marginal benefit shrinking, not reversing), boosting keeps actively reducing training error round after round, and if left unconstrained for too many rounds with too large a learning rate, it will eventually begin to fit noise in the training data and overfit. Shallow trees, a modest learning rate, and an appropriate stopping point (often chosen via a validation set) are the standard tools for keeping this bias-reduction machine from going too far.

13 Bagging versus Boosting

With both families now on the table, we can compare them directly. Table 2 summarizes the comparison; the paragraph that follows adds the necessary caveats.

The intuitive summary bears repeating:

Bagging: “Let’s make many independent models and average them.”

Boosting: “Let’s improve the model step by step.”

This summary is a useful conceptual simplification, but it should not be read as a strict rule. It would be an overstatement to claim that Bagging *only* reduces variance or that Boosting *only* reduces bias; in practice, both effects are present to varying degrees in both families, and the relative balance depends on the specific base learners, hyperparameters, and dataset involved. The safest general statement is the one already made in the table: Bagging’s *primary* lever is variance reduction through averaging over independent models, while Boosting’s *primary* lever is bias reduction through sequential, targeted correction.

Figure 2 contrasts the two training topologies directly.

14 Random Forest versus Gradient Boosting

Random Forests and Gradient Boosting are the two tree-ensemble methods most commonly used in practice, and it is worth comparing them on the dimensions that matter for actually choosing between them.

Table 2: Bagging versus Boosting.

	Bagging	Boosting
Training procedure	Trees trained independently, in parallel	Trees trained sequentially, each depending on the last
Dependence between learners	None during training	Strong: each learner targets the current ensemble’s errors
Primary intuition	“Many independent opinions, averaged”	“Improve the model step by step”
Typical effect on variance	Reduces variance substantially	Can reduce variance somewhat, but not the primary mechanism
Typical effect on bias	Little effect on bias	Reduces bias substantially, especially with more rounds
Parallelization	Naturally parallel (trees are independent)	Inherently sequential (each round needs the last)
Sensitivity to noisy labels	Relatively robust	Can be more sensitive, since mislabeled points keep receiving corrective attention
Key hyperparameters	Number of trees, tree depth, feature subsampling	Number of rounds, learning rate, tree depth, regularization
Interpretability	Individual trees remain simple; ensemble is a black box	Individual trees remain simple; ensemble is a black box
Typical use cases	High-variance base learners; tabular data with many noisy features	Tasks needing high predictive accuracy on tabular data, given careful tuning

- **Training strategy:** Random Forest trees are grown independently and can be trained in parallel; Gradient Boosting trees are grown one at a time, each depending on the fitted values of all previous trees.
- **Randomness:** Random Forest relies on two explicit randomization mechanisms (bootstrap sampling and random feature selection at each split) to decorrelate its trees. Gradient Boosting, in its basic form, relies on no such randomness at all—its diversity across trees comes from sequentially fitting different residuals, not from randomized inputs—though popular variants add subsampling of rows and/or columns at each round, partly inspired by the Random Forest idea, for both regularization and speed.
- **Feature handling:** both methods handle a mix of numeric and categorical (once appropriately encoded) features reasonably well, and both provide feature-importance measures based on how often and how effectively a feature is used for splitting.
- **Computational characteristics:** Random Forest training parallelizes trivially

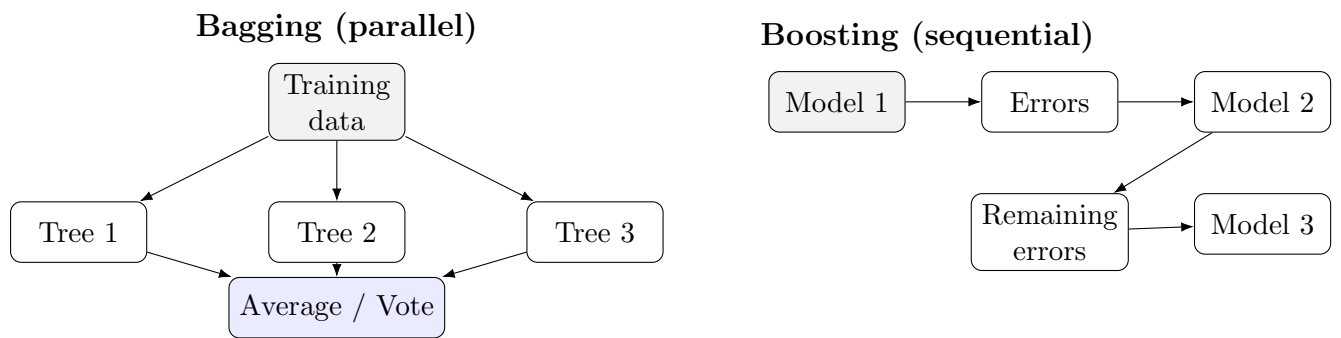


Figure 2: Bagging trains trees independently and combines them in parallel. Boosting trains trees sequentially, with each new tree targeting the errors left behind by the ensemble built so far.

across trees; Gradient Boosting’s inherently sequential structure limits parallelization across rounds, though the search for the best split within a single tree can still be parallelized.

- **Hyperparameters:** Random Forest has relatively few hyperparameters (number of trees, tree depth, number of features considered per split) and is comparatively forgiving to default settings. Gradient Boosting has more hyperparameters (number of rounds, learning rate, tree depth, subsampling rates, regularization terms) and typically requires more careful tuning to reach its best performance.
- **Robustness:** because it averages independent trees, Random Forest tends to be relatively robust to mislabeled or noisy training points and comparatively resistant to overfitting as more trees are added. Gradient Boosting, because it repeatedly refits residuals, can be more sensitive to noisy labels or outliers unless properly regularized.
- **Interpretability:** both are ensemble “black boxes” relative to a single tree, though both support post-hoc interpretation tools (feature importances, partial dependence plots, and related techniques).
- **Typical performance patterns:** with careful tuning, Gradient Boosting (and its modern variants) frequently achieves somewhat higher predictive accuracy on structured/tabular data, while Random Forest often achieves strong performance with substantially less tuning effort and lower risk of overfitting if left with default settings.

There is no universally superior method between the two. Which one performs better on a given problem depends on the characteristics of the data (signal-to-noise ratio, presence of outliers, feature redundancy), the amount of tuning effort available, and

computational constraints. In practice, both are frequently tried, and the choice is often settled empirically via cross-validation rather than by a rule of thumb.

15 XGBoost and Modern Boosted Trees

The Gradient Boosting framework of Section 11 leaves considerable freedom in how each tree is grown and how strongly the ensemble should be regularized. **XGBoost** (Extreme Gradient Boosting), introduced by Chen and Guestrin [5], is a widely used, highly optimized implementation of gradient-boosted trees that adds several refinements on top of the basic recipe.

- **Gradient and second-order information:** rather than using only the first derivative (the gradient) of the loss to decide how each tree should correct the ensemble, XGBoost also uses the second derivative (the Hessian), which provides curvature information about the loss surface and leads to more precise updates at each round.
- **Regularization:** XGBoost explicitly penalizes tree complexity—for instance, the number of leaves and the magnitude of the leaf predictions—as part of the objective being optimized, rather than relying solely on early stopping or shallow trees to control overfitting.
- **Shrinkage:** the same learning-rate idea from Section 11 is retained, scaling down each new tree’s contribution.
- **Subsampling:** rows (observations) and/or columns (features) can be randomly subsampled at each boosting round, echoing the Random Forest idea and providing both a regularization effect and a computational speedup.
- **Efficient computation:** engineering optimizations, such as smart handling of sparse data and cache-aware algorithms for finding good splits, allow XGBoost to train large ensembles substantially faster than naive implementations of the same mathematical idea.

Formally, XGBoost optimizes a **regularized objective** of the form

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (11)$$

where $l(y_i, \hat{y}_i)$ is a differentiable loss comparing the true label y_i to the predicted value $\hat{y}_i$ (exactly the loss function role played by L in Section 11), and $\Omega(f_k)$ is a penalty on the complexity of the k -th tree in the ensemble. **The purpose of the second term is worth dwelling on.** Without it, the objective would only ever reward trees for reducing training error, which—as Section 12 discussed—will eventually push the ensemble toward overfitting. The regularization term $\Omega(f_k)$ pushes back against this

tendency by penalizing trees that are unnecessarily large or that produce unnecessarily extreme leaf values, favoring simpler corrections whenever they explain the data almost as well as a more complex one would. In this sense, Equation (11) makes explicit and mathematically formal a tension that basic Gradient Boosting only manages indirectly, through the learning rate and through limiting the number of rounds.

We will not turn this section into an implementation manual; the point to take away is conceptual: XGBoost (and related modern libraries such as LightGBM and CatBoost) remain, at their core, instances of the Gradient Boosting recipe of Section 11—fitting an additive model of trees, each correcting the residual behavior of the ensemble so far—augmented with more careful regularization, more information per step (via second-order derivatives), and substantially more efficient computation.

16 A Unified Mathematical View

It is useful, having built up each method individually, to see all of them side by side.

$$\text{Single tree:} \quad \hat{y} = T(x) \tag{12}$$

$$\text{Bagging:} \quad \hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x) \tag{13}$$

$$\text{Random Forest:} \quad \hat{y} = \text{Aggregate}(T_1(x), \dots, T_B(x)), \quad \text{with bootstrap sampling and random feature selection at each split} \tag{14}$$

$$\text{Boosting:} \quad F_M(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x) \tag{15}$$

Looking at these four expressions side by side, the essential difference between the two families is not simply “many trees versus many trees”—all four involve combining trees in one way or another. The essential difference is *the relationship between the trees*.

- In Bagging and Random Forest, the trees $T_1(x), \dots, T_B(x)$ are **parallel, independent contributors**. Each tree is trained without any knowledge of the others, and the ensemble’s structure is a simple aggregation—an average or a vote—across contributors that could, in principle, be computed simultaneously.
- In Boosting, the terms $h_1(x), \dots, h_M(x)$ are **sequential corrections**. Each $h_m(x)$ is trained specifically in response to the shortcomings of $F_{m-1}(x)$, the sum of everything that came before it, and the terms cannot be computed independently of one another.

This is the deepest sense in which the four-sentence story from the introduction is literally true, and not just a simplification for beginners: a single tree makes one prediction;

Bagging asks many independent trees for their opinions and averages them; Random Forest makes those independent opinions more diverse by restricting each split’s view of the features; and Boosting abandons independence altogether in favor of a chain of targeted corrections.

17 Practical Guidance

The following checklist summarizes practical starting points; it is not a substitute for validating choices empirically on the problem at hand.

- **Start with Random Forest** when a strong, robust baseline is needed quickly, when extensive hyperparameter tuning is not feasible, or when resistance to overfitting and mislabeled data is a priority.
- **Move to Gradient Boosting** when squeezing out additional predictive accuracy matters more than training speed or ease of tuning, and when there is time and data (e.g., a validation set) available to tune the learning rate, tree depth, and number of rounds carefully.
- **Consider XGBoost (or a similar modern library)** when working with large tabular datasets where computational efficiency, built-in regularization, and support for missing values and sparse features are all valuable, and where careful tuning of a larger hyperparameter set is worthwhile.

17.1 Key Hyperparameters

For **Random Forest**: the number of trees B (more is generally safe, subject to diminishing returns and computation time), the maximum depth of each tree (deeper trees fit more but risk higher individual variance), the minimum number of samples required per leaf or per split (larger values act as a form of regularization), and the number of features considered at each split, m (smaller m decorrelates trees more but weakens each individual tree).

For **boosting methods**: the number of estimators or rounds M , the learning rate η , the maximum depth of each individual tree, the row and/or column subsampling fraction per round, and explicit regularization terms (such as the $\Omega(f_k)$ penalty of Equation (11)) that penalize tree complexity directly.

17.2 The Learning-Rate/Number-of-Trees Tradeoff

A smaller learning rate η makes each individual tree’s contribution smaller, which generally requires a larger number of rounds M to reach a comparable level of fit to the training data. In exchange, no single tree is allowed to dominate the final ensemble,

which tends to produce smoother, better-generalizing models. This tradeoff is one of the most consistently useful levers in practice: a common strategy is to fix a small learning rate (e.g., $\eta = 0.01$ to 0.1) and then tune the number of rounds against a validation set, often using early stopping to halt training once validation performance stops improving.

18 Conclusion

We began with a question: how can many imperfect decision trees be turned into a model more reliable than any single tree? The answer, it turns out, depends entirely on how the trees relate to one another.

A single decision tree is one opinion—flexible and interpretable, but liable to be led astray by whatever particular sample of data it happened to see. Bagging asks many independent trees for their opinions, grown on different bootstrap resamples of the same data, and averages them; Equation (3) tells us precisely why this helps, and precisely why it helps only so much, since averaging can cancel a tree’s individual noise but cannot cancel whatever error all the trees share. Random Forest pushes this idea further, making the trees’ opinions more genuinely diverse by hiding most of the candidate features from each individual split, which lowers the shared correlation between trees and, through the same equation, further reduces the variance of the ensemble. Boosting takes an entirely different approach: rather than collecting independent opinions, it builds an argument step by step, with each new tree correcting specifically what the current model still gets wrong—first by reweighting misclassified points, as in AdaBoost, and more generally by fitting the negative gradient of an arbitrary loss function, as in Gradient Boosting and its modern, heavily engineered descendant, XGBoost.

None of these methods is universally best. The right choice depends on the data, the computational budget, and the tuning effort available—but the reasoning in this paper should make clear *why* each method behaves the way it does, rather than leaving that behavior as an unexplained empirical fact.

References

- [1] Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [3] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.

- [4] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
- [6] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.